Phase #2 - Front End Rapid Prototyping (CSCI 3060U)

Date: *February 13th 2026*
Group Members:

- Aliseena Ahmar (100870078)
- Haider Saleem (100870637)
- Samir Ahmadi (100916280)
- Smit Kalathia (100871659)

1. Introduction & Purpose

This document describes the design and initial implementation of the Front End ATM system developed for the CSCI 3060U course project. The system is a console-based banking application that processes a stream of banking transactions, produces user-visible responses, and records daily transactions to a file.

This phase focuses on rapid prototyping and architectural clarity, rather than full correctness or exhaustive testing. The system is designed to align with the requirements analyzed in Phase 1 and will be fully validated using the established requirements tests in later phases of the project.

2. Program Usage & I/O Contract

2.1 Program Invocation

The Front End application is executed from the command line as follows:

Compile the Program

From the project root:

javac -d phase2\bin phase2\src*.java

This compiles all source files into the phase2\bin directory.

Run the Program Interactively

java -cp phase2\bin AtmApp phase2\data\currentaccounts.txt out.atf

- The program will wait for transaction input from standard input.
- Type transactions manually or provide input using redirection.

2.2 Inputs

Input Source	Description
Standard Input (stdin)	Stream of banking transactions, one transaction code per line followed by required data fields
currentaccounts.txt	Fixed-length text file containing all current bank account records

2.3 Outputs

Output Source	Description
Standard Output (stdout)	User-visible success and error messages
out.atf	Daily transaction file written when a logout occurs

All interaction is strictly text-based to support automated testing and scripting. No graphical or interactive prompts are used.

3. High-Level Architecture Overview

The Front End system is structured into a small set of clearly defined components, each with a single responsibility.

- **AtmApp**
  Acts as the entry point of the application. It parses command-line arguments, initializes core components, and starts the transaction processing loop.

- **TransactionProcessor**
  Controls the main execution flow. It reads transaction codes from standard input, enforces session rules, validates inputs, and dispatches the appropriate transaction handlers.

- **AccountsRepository**
  Loads bank account data from the current accounts file and provides lookup and validation

services for accounts.

- **SessionContext**
  Maintains the state of the current session, including whether a user is logged in, session mode (standard or admin), and session-based limits.

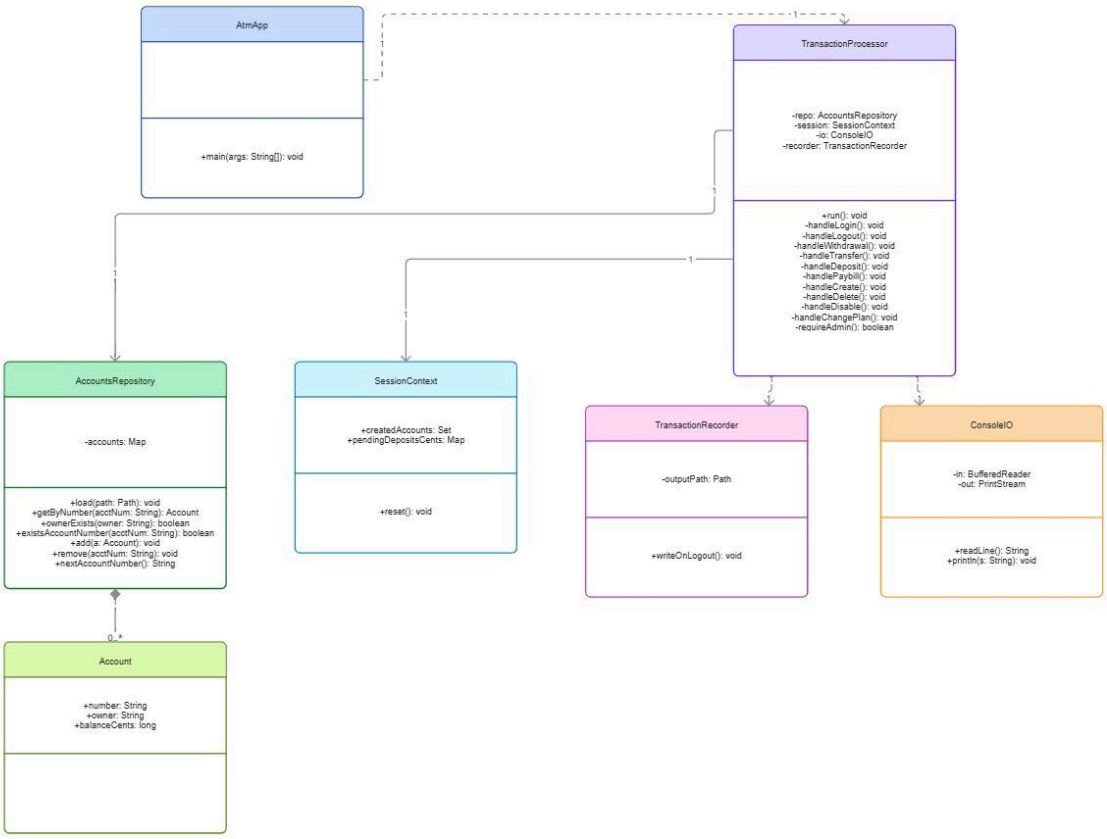
- **Account**
  Represents a single bank account, including its account number, holder name, status, plan, and balance.

- **Messages**
  Centralizes all user-visible output messages to ensure consistency with requirements tests.

This separation of concerns results in a system that is easy to understand, maintain, and extend.

4. UML Class Diagram

The following UML class diagram illustrates the structure of the Front End system and the relationships between its components



5. Class and Method Intention Table

Class / Method	Responsibility
AtmApp	Entry point of the application
main(String[] args)	Parses command-line arguments and starts the Front End
TransactionProcessor	Coordinates transaction processing and validation
TransactionProcessor(ConsoleIO io, AccountsRepository repo, TransactionRecorder recorder)	Constructs processor with I/O, repository, and recorder dependencies
run()	Reads transaction stream and dispatches handlers
handleLogin()	Starts a standard or admin session
handleLogout()	Ends a session and triggers transaction file output
handleWithdrawal()	Validates and processes withdrawals
handleTransfer()	Validates and processes transfers
handleDeposit()	Validates and processes deposits (may record pending deposits for later)
handlePaybill()	Validates and processes bill payments
handleCreate()	Creates a new account (admin session)
handleDelete()	Deletes an existing account (admin session)
handleDisable()	Disables an account (admin session)
handleChangePlan()	Changes an account plan (admin session)

requireAdmin()	Enforces admin-only access for privileged transactions
AccountsRepository	Loads and manages bank account data
load(Path path)	Reads accounts from the current accounts file
getByNumber(String acctNum)	Locates an account by account number
ownerExists(String owner)	Checks whether an owner exists in the repository
existsAccountNumber(String acctNum)	Checks whether an account number exists
add(Account a)	Adds a new account to the repository
remove(String acctNum)	Removes an account from the repository
nextAccountNumber()	Generates the next available account number
Account	Stores account data and state (account number, owner, balance, status/plan)
Account(String number, String owner, Status status, Plan plan, long balanceCents)	Constructs an account record
AccountStatus (enum)	Defines possible account states (present as a separate enum file)
SessionContext	Tracks session state, created accounts, and pending deposits
reset()	Clears session state and per-session data
SessionMode (enum)	Identifies session type (standard vs admin)

TransactionRecorder	Writes the daily transaction file (.atf)
TransactionRecorder(Path out)	Stores output path for transaction file
writeOnLogout()	Creates or overwrites the daily transaction file on logout (prototype output)
ConsoleIO	Handles reading from stdin and writing to stdout
ConsoleIO(BufferedReader in, PrintStream out)	Wraps standard input/output streams
readLine()	Reads a line from input
println(String s)	Prints a line to output
Messages	Constants container for user-visible output strings

6. Design Decisions and Constraints

Several design decisions were made to support clarity and testability:

- All input and output is text-based to allow automated testing using input and output redirection.
- Output messages are centralized in a single class to prevent inconsistencies between expected and actual outputs.
- Session state is encapsulated to avoid scattered conditional logic.
- The design favors simple control flow and explicit logic over advanced patterns or optimizations.

These decisions support maintainability and align with Agile development principles.

7. Known Limitations (Phase 2 Scope)

As this is a rapid prototype, the following limitations are acknowledged:

- Not all edge cases have been fully validated.
- Some error-handling behavior will be refined during full testing in later phases.